

Write-up CTF : ecbcbcwtf

1. Description du Challenge

- **Catégorie** : Cryptographie
- **Difficulté** : Facile / Moyen
- **Objectif** : Récupérer le drapeau secret (**FLAG**) protégé par l'algorithme AES, en exploitant une mauvaise configuration des modes de chiffrement du serveur.

2. Analyse du Code Source

En analysant le code Python (Flask) fourni, on remarque deux fonctions principales exposées sur le réseau :

1. **/ecbcbcwtf/encrypt_flag/** : Cette fonction chiffre le **FLAG** en utilisant le mode **AES-CBC** avec une clé secrète (**KEY**) et un vecteur d'initialisation aléatoire (**IV**). Elle renvoie l'IV collé au texte chiffré.
2. **/ecbcbcwtf/decrypt/<ciphertext>/** : Cette fonction agit comme un oracle de déchiffrement. Cependant, elle contient une vulnérabilité critique : elle déchiffre le texte fourni en utilisant le mode **AES-ECB** au lieu du mode CBC d'origine.

La Faille (Théorie)

En mode **CBC**, le déchiffrement d'un bloc C_i pour obtenir le texte clair P_i se déroule en deux étapes :

1. Déchiffrement AES brut du bloc chiffré (ce qui correspond au mode **ECB**).
2. Opération **XOR** entre le résultat de ce déchiffrement et le bloc chiffré précédent. Pour le tout premier bloc, le XOR se fait avec l'**IV**.

Puisque l'oracle du serveur nous donne directement le résultat de l'étape 1 (le déchiffrement ECB), il nous suffit de récupérer ces valeurs intermédiaires et de **calculer l'étape 2 (le XOR) manuellement**.

3. Résolution pas à pas

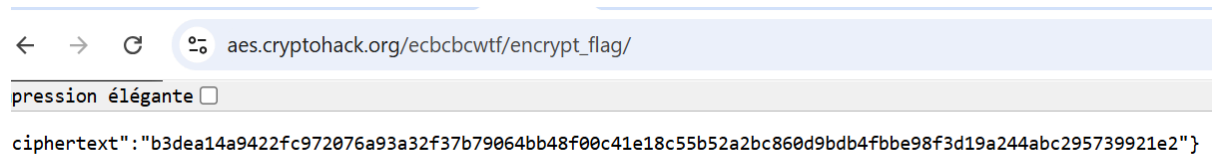
Étape 1 : Récupération du Ciphertext

On interroge d'abord l'API pour obtenir le flag chiffré :
https://aes.cryptohack.org/ecbcbcwtf/encrypt_flag/

Réponse obtenue :

```
{"ciphertext":"b3dea14a9422fc972076a93a32f37b79064bb48f00c41e18c55b52a2bc860d9bdb4fbbe98f3d19a244abc295739921e2"}
```

Capture :



Étape 2 : Découpage en blocs

Les blocs AES font 16 octets, soit **32 caractères hexadécimaux**. On découpe la chaîne reçue :

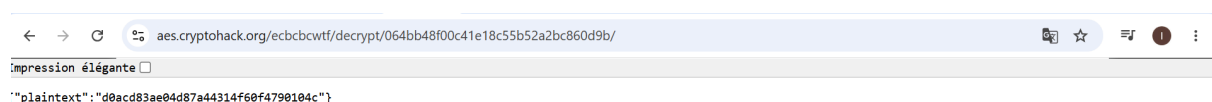
- **IV** : b3dea14a9422fc972076a93a32f37b79
- **Bloc Chiffré 1** : 064bb48f00c41e18c55b52a2bc860d9b
- **Bloc Chiffré 2** : db4fbbe98f3d19a244abc295739921e2

Étape 3 : Interrogation de l'Oracle (Déchiffrement ECB)

On envoie séparément le bloc 1 et le bloc 2 à la route de déchiffrement :

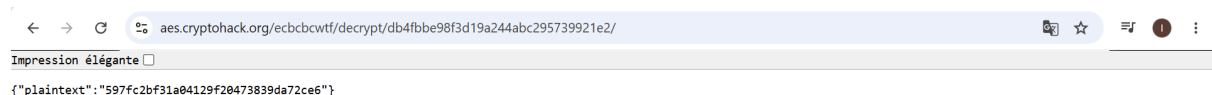
- **Pour le Bloc 1** :
/ecbcbctf/decrypt/064bb48f00c41e18c55b52a2bc860d9b/
 - *Résultat brut* : d0acd83ae04d87a44314f60f4790104c

Capture:



- **Pour le Bloc 2** :
/ecbcbctf/decrypt/db4fbbe98f3d19a244abc295739921e2/
 - *Résultat brut* : 597fc2bf31a04129f20473839da72ce6

Capture:



Étape 4 : XOR Calculator et conversion

Étape 4 : Calcul du XOR et conversion en texte clair (Méthode Manuelle)

Pour reconstituer la logique d'enchaînement du mode CBC sans script, nous appliquons manuellement l'opération XOR entre les blocs grâce à un outil en ligne (*XOR Calculator*), puis nous convertissons le résultat hexadécimal en texte lisible via *RapidTables*.

1. Déchiffrement du premier bloc (Bloc1)

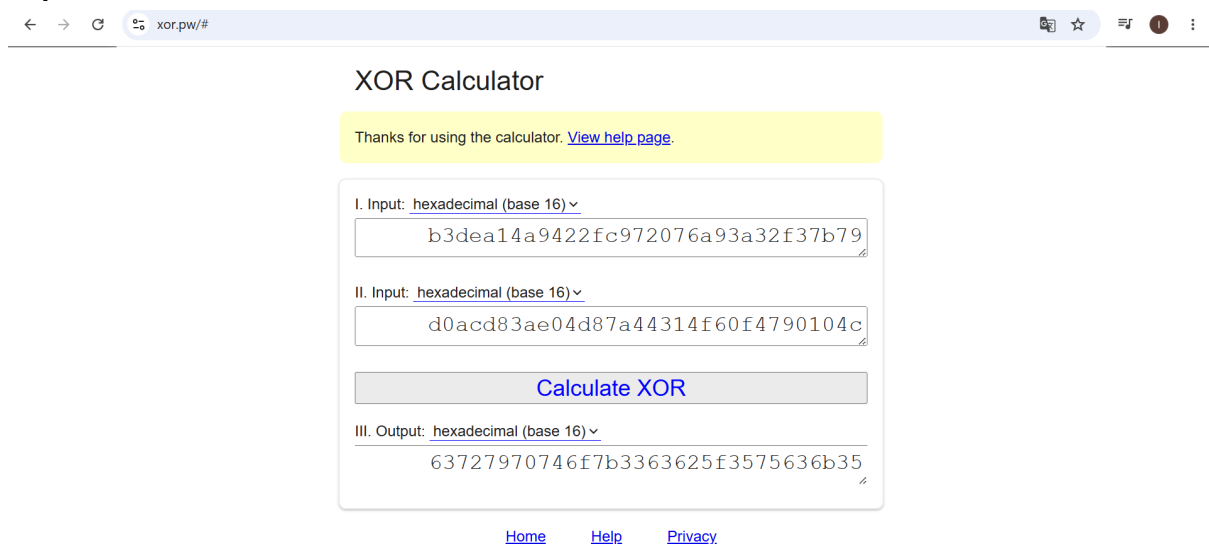
D'après la spécification du mode CBC, le premier bloc déchiffré doit subir un XOR avec le vecteur d'initialisation (IV) :

- Valeur 1 (Déchiffrement brut 1) : **d0acd83ae04d87a44314f60f4790104c**
- Valeur 2 (IV) : **b3dea14a9422fc972076a93a32f37b79**

En insérant ces deux chaînes dans le *XOR Calculator*, on obtient le résultat hexadécimal suivant :

63727970746f7b3363625f3575636b35

Capture:



The screenshot shows a web browser window with the address bar displaying 'xor.pw/#'. The page title is 'XOR Calculator'. A yellow message box says 'Thanks for using the calculator. [View help page.](#)'. The main content area has three sections: 'I. Input: hexadecimal (base 16) ~' with the value 'b3dea14a9422fc972076a93a32f37b79', 'II. Input: hexadecimal (base 16) ~' with the value 'd0acd83ae04d87a44314f60f4790104c', and a 'Calculate XOR' button. Below the button, 'III. Output: hexadecimal (base 16) ~' shows the result '63727970746f7b3363625f3575636b35'. At the bottom, there are links for 'Home', 'Help', and 'Privacy'.

En collant ce résultat dans le convertisseur Hex-vers-ASCII de *RapidTables*, on découvre la première partie du flag :

crypto{3cb_5uck5

RapidTables

Home > Conversion > Number conversion > ASCII,hex,binary,decimal,base64 converter

ASCII, Hex, Binary, Decimal, Base64 converter

Enter [ASCII text](#) or hex/binary/decimal numbers:

Open File

× Reset

Number delimiter

Space

☐ 0x/0b prefix

ASCII text

crypto{3cb_5uck5

Hex (bytes)

63727970746f7b3363625f3575636b35

2. Déchiffrement du second bloc (Bloc 2)

Pour le deuxième bloc, la règle du CBC veut qu'on applique un XOR avec le bloc chiffré qui le précédait directement dans la chaîne d'origine :

- **Valeur 1 (Déchiffrement brut 2) :** 597fc2bf31a04129f20473839da72ce6
- **Valeur 2 (Bloc Chiffré 1) :** 064bb48f00c41e18c55b52a2bc860d9b

Le calcul XOR nous donne le second résultat hexadécimal :

5f34763031645f31375f21212121217d

← → ↻ % xor.pw/#

XOR Calculator

Thanks for using the calculator. [View help page.](#)

I. Input: hexadecimal (base 16) ▾
597fc2bf31a04129f20473839da72ce6

II. Input: hexadecimal (base 16) ▾
64bb48f00c41e18c55b52a2bc860d9b

Calculate XOR

III. Output: hexadecimal (base 16) ▾
5f34763031645f31375f212121217d

[Home](#) [Help](#) [Privacy](#)

Après traduction sur *RapidTables*, on obtient la fin du flag :

`_4v01d_17_!!!!}`

← → ↻ % rapidtables.com/convert/number/ascii-hex-bin-dec-converter.html

RapidTables

Home > Conversion > Number conversion > ASCII,hex,binary,decimal,base64 converter

ASCII, Hex, Binary, Decimal, Base64 converter

Enter [ASCII text](#) or hex/binary/decimal numbers:

Number delimiter
Space ▾

☐ 0x/0b prefix

ASCII text
_4v01d_17_!!!!}

Hex (bytes)
5f34763031645f31375f212121217d

Le flag recherché est donc: **crypto{3cb_5uck5_4v01d_17_!!!!}**